

NEPAL COLLEGE OF INFORMATION TECHNOLOGY
Affiliated to Pokhara University

CASE STUDY ON

Smart ATM Security System
Using IoT and Face Recognition

Course Name: Internet of Things

Course Code: CMP370

Program: B.E. Information Technology

Semester: Sixth Semester

Submitted By: Suraj Kumar Jha

Roll Number: 231537

Submitted To: Er. Rishi K. Marseni

Date: 11 July 2026



Department of Information Technology

NEPAL COLLEGE OF
INFORMATION TECHNOLOGY

Balkumari, Lalitpur, Nepal

Table of Contents

1. Introduction	3
2. Background Study	3
3. Problem Statement	4
4. System Architecture.....	4
5. Hardware Components	6
6. Software Components	7
7. Communication Protocols.....	7
8. Data Flow Analysis	8
9. Security Analysis	9
10. Mapping with Course Labs.....	10
11. Challenges and Limitations	10
12. Future Improvements	11
13. Personal Reflection	11
14. Conclusion	11
15. References	11

1. Introduction

The Internet of Things (IoT) is a network of physical devices, sensors, and actuators embedded with processing power and connectivity, allowing them to collect, exchange, and act on data with minimal human intervention. Over the past decade IoT has moved from research concept to foundational technology across healthcare, agriculture, manufacturing, and finance. One of its most security-sensitive applications lies in banking, specifically at Automated Teller Machines (ATMs), which handle billions of unattended cash transactions worldwide every year.

This case study examines a **Smart ATM Security System using IoT and Face Recognition**, which upgrades the traditional card-plus-PIN model by adding a real-time biometric identity check. An embedded camera and edge-processing unit continuously verify that the person operating the ATM is the actual cardholder; on mismatch, the system autonomously triggers an alarm and alerts bank security staff through a cloud-connected dashboard.

The motivation is a well-documented global problem: card fraud, stolen or cloned cards, shoulder-surfed PINs, cardholders forgetting their card keeps causing losses despite decades of password-based security. Banking is now shifting toward multi-factor and biometric authentication, and IoT-enabled edge AI makes real-time on-site verification feasible rather than relying only on after-the-fact CCTV review. This topic is important because it merges IoT, computer vision, and cloud computing into one practical solution, making it a strong case study for the architecture, protocol, and security concepts covered in this course.

2. Background Study

ATMs have historically relied on a "one card, one PIN" model that authenticates the card and PIN but never verifies that the person entering the PIN is the legitimate cardholder. Existing CCTV setups mostly serve as after-the-fact evidence footage is manually reviewed only once a fraud complaint is filed, by which time the money is already lost [1].

Prior research has explored closing this gap with face recognition. Jacintha et al. proposed an IoT-based ATM surveillance system combining a face-recognizing camera with tilt/vibration sensors to flag both identity mismatch and physical tampering [1]. Patoliya and Desai implemented a face-detection ATM security system on an embedded Linux platform using a Raspberry Pi, showing that low-cost single-board computers can run real-time face detection at the edge [2]. Other work combines facial recognition with liveness cues such as eye-blink detection to resist photo-based spoofing under the uncontrolled lighting typical of ATM kiosks [3].

On the algorithm side, deep-learning face embeddings significantly outperform older hand-crafted features. The DeepID2 architecture, which jointly optimizes identification and verification signals during

training, was shown to substantially reduce false-match and false-reject rates versus earlier CNN-based embeddings [4].

Despite this body of work, most solutions still fall short in three areas: the enrolled face image is often a stale ID photo never updated; preprocessing for harsh, variable ATM lighting is frequently skipped; and most implementations verify locally with no structured cloud pipeline for logging, alerting, or database updates. A layered IoT architecture device, network, cloud is well suited to close exactly this gap, which is the approach taken here.

3. Problem Statement

Traditional ATM authentication verifies **possession** (the card) and **knowledge** (the PIN) but never verifies **identity** whether the person at the machine is the cardholder. This gap is exploited through stolen/cloned cards, phished PINs, and cardholders forgetting their card. CCTV only provides evidence after a crime, offering no real-time prevention.

The Smart ATM Security System solves this by adding a real-time, IoT-connected face verification layer at the point of transaction, comparing a live image against a continuously updated cardholder record and raising an alarm on mismatch before cash is dispensed. This matters because ATM fraud causes direct financial loss and erodes trust in banking infrastructure at scale. Beneficiaries include cardholders (protected from losses), banks (reduced fraud liability), and law enforcement (automated flagging instead of manual video review).

4. System Architecture

The system follows the standard three-layer IoT architecture from Unit 2: Device, Network, Cloud adapted for real-time biometric verification (Fig. 1).

Device Layer: An IR proximity sensor at the card slot detects insertion and triggers the camera to capture a live image. A Raspberry Pi 4 acts as the edge node: it runs preprocessing (noise filtering, histogram equalization for variable kiosk lighting), a cascaded CNN for face detection, and a DeepID2-based embedding model for verification the same detect-then-verify pipeline used in prior ATM research [2], on modern edge hardware. On mismatch, the Pi drives a relay/buzzer to sound a local alarm and can hold the transaction.

Network Layer: The Pi connects over WiFi/Ethernet to the backend. Verification requests use a **REST API** over HTTPS, since this traffic is transactional request–response and benefits from REST's simplicity and statelessness. Alarm notifications instead use **MQTT** with a publish–subscribe model, since its lower overhead suits urgent, low-latency alerts better than a polled REST endpoint.

Cloud Layer: An AWS EC2 instance hosts a FastAPI backend that runs the final match decision, logs transactions, and returns an allow/deny signal. A PostgreSQL database stores face embeddings and transaction/alert logs. A Chart.js dashboard gives security staff a live view of verification confidence and alarms across all connected ATMs.

This separation is deliberate: the device layer keeps basic detection fast and network-independent; the network layer moves only minimal data; the cloud layer centralizes storage, auditing, and monitoring across the whole ATM network rather than each machine operating in isolation.

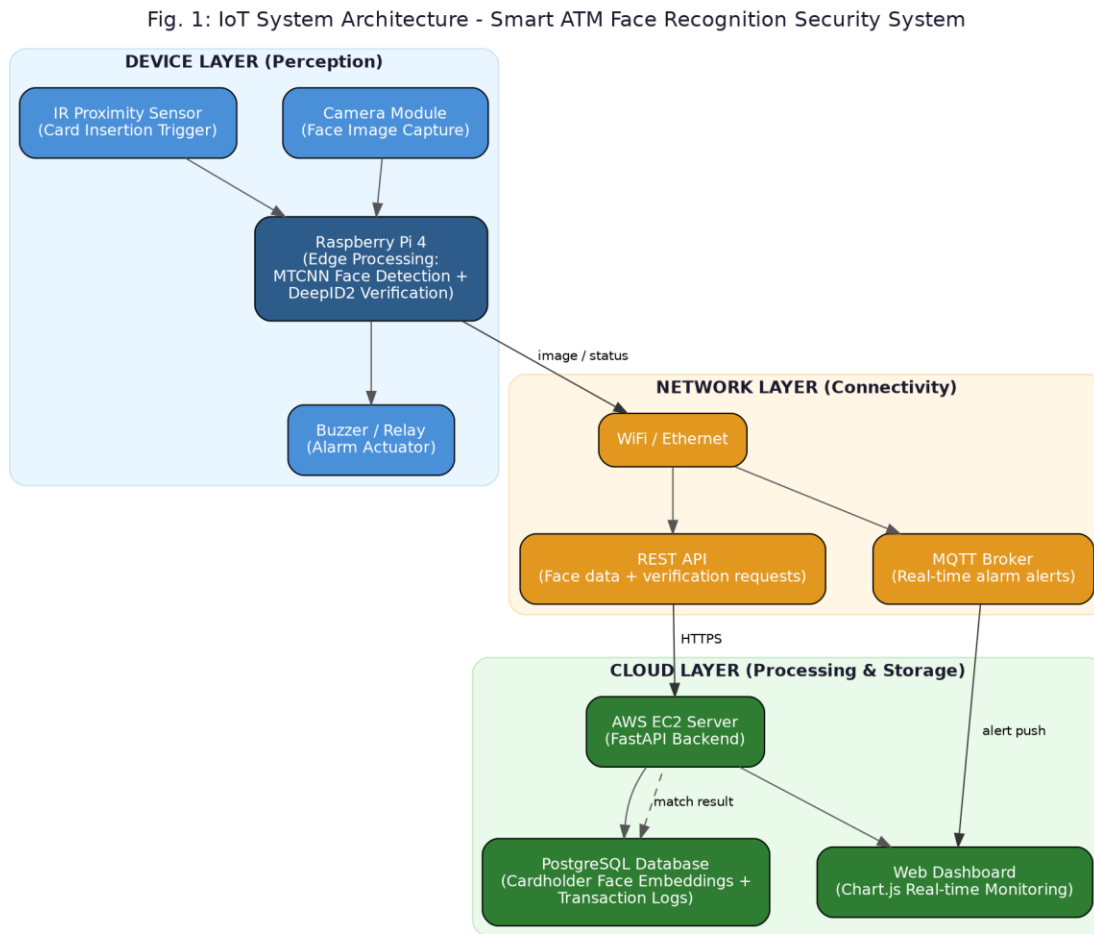


Fig. 1: IoT System Architecture — Smart ATM Face Recognition Security System

Fig. 3: System Block Diagram

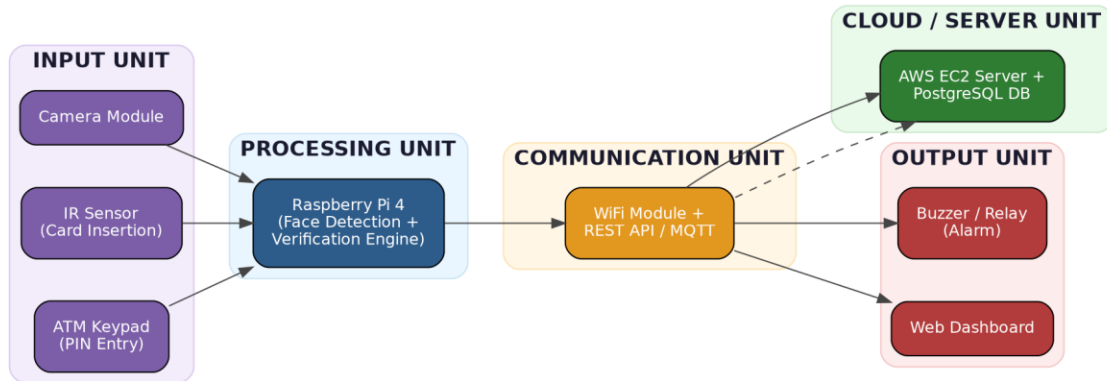


Fig. 3: System Block Diagram

5. Hardware Components

Raspberry Pi 4 (Model B): the core processing unit, using a quad-core Cortex-A72 SoC, up to 8 GB RAM, dual-band WiFi, Gigabit Ethernet, and a 40-pin GPIO header. Its CSI camera interface and compute headroom let it run the cascaded-CNN detector and DeepID2 verification in real time, as demonstrated in comparable embedded-Linux ATM research [2].

Advantages: low cost, full Linux OS with standard Python/OpenCV support, active ecosystem.

Limitation: lower throughput than a dedicated GPU/accelerator under heavy load.

Camera Module: an 8MP CSI camera, chosen for low-latency native integration with the Pi and enough resolution to preserve the facial detail needed for verification.

IR Proximity Sensor: mounted at the card slot; it emits and measures reflected IR light to detect card insertion and trigger capture. Simple and low-power, but accuracy can be affected by ambient IR interference such as direct sunlight.

Actuators: Relay and Buzzer: the relay is an electromechanical switch letting a low-power GPIO output safely control a higher-current alarm circuit; the buzzer/siren is the audible output. The same relay line can also hold the transaction or trigger a physical lock, giving direct enforcement, not just a passive alert.

Fig. 5: Sensor Connection Diagram (Camera + IR Sensor to Raspberry Pi 4)

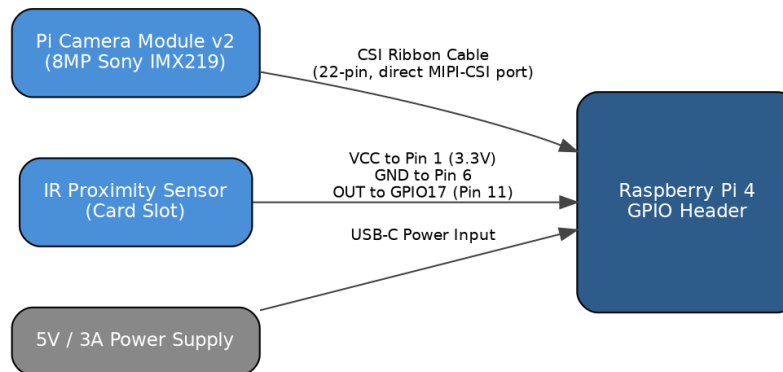


Fig. 5: Sensor Connection Diagram (Camera + IR Sensor to Raspberry Pi 4)

Fig. 6: Actuator Connection Diagram (Buzzer/Relay to Raspberry Pi 4)

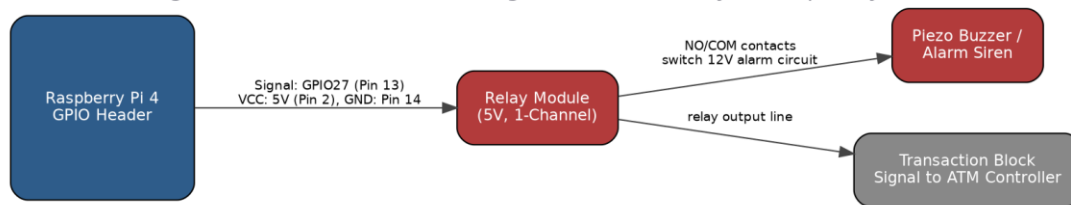


Fig. 6: Actuator Connection Diagram (Buzzer/Relay to Raspberry Pi 4)

6. Software Components

Programming Languages: Python runs on the Pi for capture, preprocessing, and the detection/verification pipeline via OpenCV and a deep-learning framework, chosen for its mature computer-vision libraries; C++ is used selectively where inference speed matters most.

Frameworks: The backend uses **FastAPI**, chosen over Flask for native async support (useful for concurrent requests from many ATMs) and automatic API documentation, simplifying the REST integration in Section 4.

Database: **PostgreSQL** is used in production for strong consistency and scalability across a large ATM network; **SQLite** is used locally during development and lab testing before deployment.

Dashboard: Built with **HTML/JavaScript** and **Chart.js**, showing live verification-confidence charts and a transaction/alert table, refreshed via REST polling or WebSocket push.

Cloud Platform: **AWS EC2** hosts the FastAPI backend and dashboard, directly extending the Lab 1 hosting exercise, keeping the whole pipeline inside one controllable environment rather than a telemetry-only

platform like ThingSpeak, which suits simple sensor dashboards more than a transactional, security-critical workload.

Fig. 4: Web Dashboard Mockup (Bank Security Monitoring Interface)

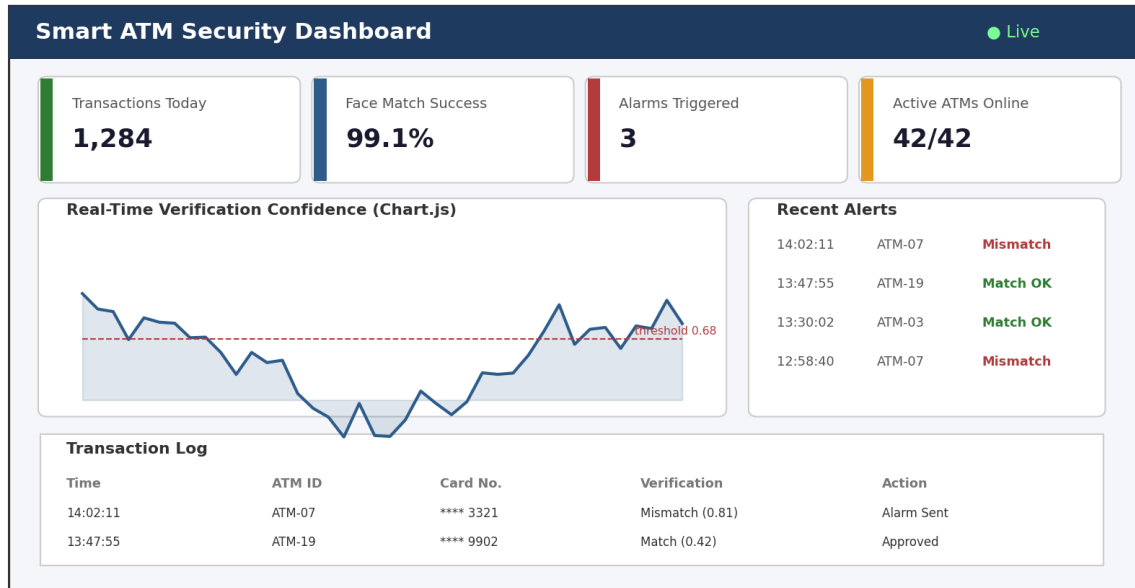


Fig. 4: Web Dashboard Mockup (Bank Security Monitoring Interface)

7. Communication Protocols

- **HTTP/REST API:** stateless request-response, used for enrollment and verification requests. Easy to secure with HTTPS/TLS, but carries more per-message overhead than MQTT [5].
- **MQTT:** lightweight publish-subscribe, used for alarm notifications since its smaller headers and near-real-time delivery suit urgent alerts better than REST polling [6], [7].
- **CoAP:** UDP-based, common in constrained sensor networks; not the primary choice here (the ATM is not power-constrained), but a candidate for a future battery-backed backup module.
- **WebSocket:** used between EC2 and the dashboard for live push updates without repeated polling.

Criterion	HTTP/REST	MQTT	CoAP
Speed	Moderate overhead	Fast, lightweight headers	Fast (UDP)
Reliability	High (TCP)	High (QoS 0/1/2)	Lower (needs app-layer retries)
Power Consumption	Higher	Low	Very low
Scalability	Good (stateless)	Excellent (pub-sub)	Good for constrained swarms

REST is used for verification calls needing a clear request-response guarantee, while MQTT handles alarm broadcast, simultaneously notifying the dashboard, on-site guard terminal, and mobile app the instant a mismatch occurs, faster than polling [7].

8. Data Flow Analysis

On card insertion, the IR sensor triggers the camera. The frame is denoised and contrast-corrected, then passed to the detector, which locates the largest face (assumed the user). This face is verified against the cardholder's stored embedding via DeepID2, producing a similarity score against a threshold. The match result is sent via REST to the EC2 backend, which logs the transaction, updates the face record on success, and returns an allow/deny decision.

This flow builds directly on three labs: **Lab 2** provided the REST API pattern used to transmit the verification result and receive the decision; **Lab 3** provided the dashboard pattern now showing confidence trends and alarm history; **Lab 5** provided the end-to-end structure — device, network protocol, cloud service, user interface — replicated here at ATM-network scale.

On success the stored face image is replaced, keeping enrollment current and addressing the stale-image limitation from Section 2 [1]. On failure the record is untouched, the buzzer triggers, and an MQTT alert reaches the dashboard within seconds, before cash is dispensed.

Fig. 2: End-to-End Data Flow Diagram

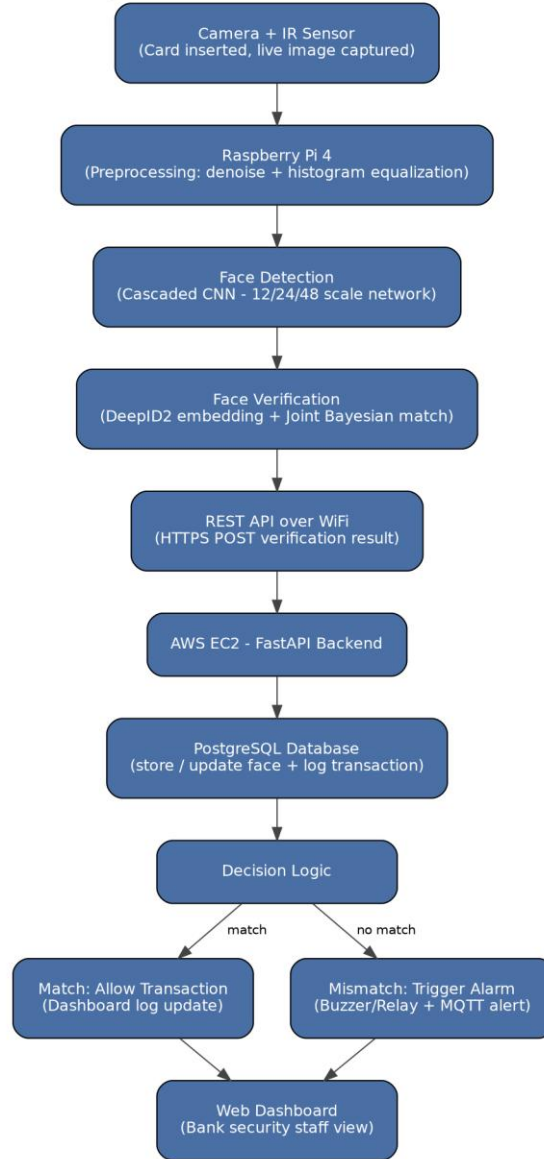


Fig. 2: End-to-End Data Flow Diagram

9. Security Analysis

Relating to Unit 6, key threats include:

- **Unauthorized Access:** attackers targeting the face database or API traffic to enumerate identities mitigated with role-based access control and firewalling on the EC2 instance.
- **Data Tampering:** altering logs or face records to hide fraud mitigated with access auditing and checksums on stored embeddings.

- **Replay Attacks:** resending a captured request to force a false match mitigated with timestamped, single-use API tokens.
- **Password/Credential Attacks:** brute-forcing the API or dashboard mitigated with rate limiting and lockout policies.

Measures implemented: HTTPS/TLS encrypts all traffic between the Pi, EC2, and dashboard; API keys authenticate each ATM device to the backend; encryption at rest protects stored embeddings and identity data, following practices recommended in IoT authentication surveys for resource-constrained ecosystems [8], [9]; and authentication/authorization layers separate what a field device can do (submit verification) from what a dashboard admin can do (query/modify records).

Because the system already combines card, PIN, and biometric factors, it aligns with the broader industry shift toward multi-factor authentication, which substantially raises the cost of successful fraud compared to single- or dual-factor systems.

10. Mapping with Course Labs

Lab 1 :- AWS EC2 hosting: The cloud layer (FastAPI + PostgreSQL + dashboard) is hosted on EC2 using the same Nginx-fronted pattern practiced in Lab 1, now serving as the production host for verification and monitoring.

Lab 2 :- REST API: Lab 2's REST design is the exact mechanism used for an ATM to submit a verification request and receive an allow/deny response, and for enrollment uploads.

Lab 3 :- Dashboard/visualization: The Chart.js dashboard from Lab 3 becomes the security monitoring interface, showing live confidence trends and alert logs as in Section 8.

Lab 4 :- Embedded devices: Lab 4's work with embedded boards, sensors, and GPIO is the direct foundation for the Pi 4 + camera + IR sensor + relay/buzzer assembly in Section 5 and Figs. 5–6.

Lab 5 :- End-to-end IoT workflow: Lab 5's full pipeline sensor to network to cloud to interface is replicated at system scale in Section 8, showing the same pattern taught for a single node scales to a security-critical, networked ATM deployment.

11. Challenges and Limitations

Key practical challenges: **internet dependency** delays cloud logging/alerts if connectivity drops, though local edge verification can still run on a cached record; **sensor/camera accuracy** suffers under poor lighting, motion blur, or occlusion (masks, angles); **security** introduces a new attack surface (API, biometric database) requiring the defenses in Section 9; **power consumption** is higher than a simple sensor node due to continuous camera and CNN inference; **cloud cost** grows with EC2 usage and data storage across

thousands of ATMs; **scalability** requires the backend to handle many concurrent requests without added latency; and **maintenance** needs periodic model retraining, camera/sensor servicing, and software updates.

12. Future Improvements

Future directions include newer face-embedding models (e.g., ArcFace) for higher accuracy and pose robustness; **edge accelerators** (Coral TPU, Jetson) to cut latency and cloud bandwidth; **5G** for faster cloud-side logging in dense deployments; **digital twins** of the ATM network to simulate and stress-test the fraud pipeline before rollout; **predictive analytics** on transaction history to flag suspicious patterns before a mismatch occurs; and **blockchain** for an immutable, tamper-evident transaction/alarm ledger, strengthening the tampering defenses from Section 9.

13. Personal Reflection

Working through this case study deepened my understanding of how the layered IoT architecture taught in this course applies to a genuinely high-stakes security problem rather than a purely academic sensor demo. Studying the ATM face-recognition patent alongside real IoT security literature showed me that even a computer-vision-heavy system still depends on the same networking and cloud fundamentals — REST, MQTT, cloud hosting — practiced in the labs. Mapping each lab to a specific part of this system made the labs feel like reusable building blocks rather than isolated exercises. This strengthened my system-level thinking, protocol selection based on tradeoffs rather than default choices, and my ability to structure a security analysis around concrete threats and mitigations. It has also increased my interest in edge AI and computer-vision security as a direction for further study.

14. Conclusion

This case study analyzed a Smart ATM Security System that strengthens card-and-PIN authentication with real-time IoT-connected face verification. Combining an edge-processing device layer, a network layer (REST for verification, MQTT for alerts), and a cloud layer (EC2, PostgreSQL, dashboard) closes a long-standing gap: verifying the user is truly the cardholder before cash is dispensed, rather than only recording evidence afterward. This is achievable with low-cost, widely available hardware and standard IoT protocols, provided preprocessing, database freshness, and end-to-end security are properly engineered. As IoT, edge AI, and cloud infrastructure mature, systems like this are likely to become a standard layer of defense across banking, reducing card fraud while giving security teams real-time, actionable visibility instead of after-the-fact review.

15. References

- [1] V. Jacintha, J. Nagarajan, K. T. Yogesh, S. Tamilarasu, and S. Yuvaraj, "An IoT based ATM surveillance system," in *Proc. 2017 IEEE Int. Conf. Computational Intelligence and Computing Research (ICCIC)*, 2017, doi: 10.1109/ICCIC.2017.8524485.
- [2] J. J. Patoliya and M. M. Desai, "Face detection based ATM security system using embedded Linux platform," in *Proc. 2017 2nd Int. Conf. for Convergence in Technology (I2CT)*, 2017, doi: 10.1109/I2CT.2017.8226097.
- [3] "Intelligent ATM ingress protection: Integrating facial recognition and liveness detection for enhanced security," IEEE Conference Publication, 2025.
- [4] Y. Sun, X. Wang, and X. Tang, "Deep learning face representation by joint identification-verification," in *Proc. 27th Int. Conf. Neural Information Processing Systems (NIPS'14)*, 2014.
- [5] K. Yokotani and Y. Sasaki, "Comparison with HTTP and MQTT on required network resources for IoT," in *Proc. 2016 Int. Conf. Control, Electronics, Renewable Energy and Communications (ICCEREC)*, 2016, doi: 10.1109/ICCEREC.2016.7814989.
- [6] HiveMQ Team, "MQTT vs. HTTP for IoT," HiveMQ Technical Blog, 2024.
- [7] IoT For All Team, "MQTT vs HTTP for IoT: Detailed protocol comparison," IoT For All, 2023.
- [8] M. A. Ferrag et al., "A comprehensive survey on authentication and secure key management in Internet of Things: Challenges, countermeasures, and future directions," Journal of Network and Computer Applications, ScienceDirect, 2023.
- [9] "A survey of security and privacy issues in the Internet of Things," e-space.mmu.ac.uk, ETT, 2020.
- [10] Guo Qian, Lin Liang, Wan Hai, and Wang Qing, "ATM identity authentication method based on face recognition," Sun Yat-Sen University, China Patent CN106339673A, Jan. 18, 2017.
- [11] Amazon Web Services, "Amazon EC2 Documentation," AWS Documentation, accessed 2026.